

Reflection

In two hours I built a single-file WebGL2 cube. It has 27 sections with a 7×7 grid of shapes on every face, an isometric camera, and flat gradient colors. It loops forever through random single, double and staggered triple layer turns, plus a corner twist. Every piece snaps back to an exact grid cell after each move, and it runs without errors.

What's missing

The color is the biggest gap. The original reads as one soft pastel gradient flowing across the whole cube, while mine gives each section its own saturated gradient, so it looks patchy. One of my prompts was ambiguous, and I didn't catch the agent's misreading of it.

The twist also doesn't match. The original swirls inside a nearly fixed hexagon, as if the dot field itself spirals. Mine rotates rigid rings and shrinks the outer one to keep it inside, which leaves the outline notched mid-move.

There are smaller gaps too. My shapes are heavier than the original's fine dots, and my background is pure white instead of off-white. I also never tuned the timing or easing against the original.

What I'd do differently

I'd turn the reference into numbers before writing a prompt. The agent couldn't see the original, so my descriptions were its only source of truth, and vague words like "swirl" and "bigger" cost me rounds of correction. I'd spend the first 20 minutes frame-stepping my recording and writing a measurable spec. It would include dots per edge, sampled colors, move durations and pauses counted in frames, easing shapes, and what the outline does during the twist.

I'd choose the architecture based on the hardest behavior, not the easiest. I let the agent commit to rigid blocks in my first prompt, which made layer turns easy and the twist nearly impossible. Because the camera never moves, the cube is effectively a 2D hexagon of dots. If each visible dot were an instance with its own position, one model could handle both rigid turns and a fluid swirl. Next time I'd ask the agent for two or three architectures with tradeoffs before any code.

I'd build the verification tools first. The agent eventually built a controllable clock, seeded randomness, grid checks and pixel measurements against the hexagon, but only mid-session, after bugs appeared. With those in place from the start, I could compare my renders against the timestamps in my spec, and "does it look right" would become a measurable difference.

I'd have the agent restate each request before building it. Asking for its interpretation and acceptance criteria before each change would have caught the gradient misreading. It would also have prevented work I later threw away: the 5×5×5 grid, the slice that pulled out before rotating, and the test case I deleted.

I'd budget my time on purpose: roughly 20 minutes on the spec, 25 on architecture and testing tools, 65 on matching the original, and 10 on polish. This time, most of my two hours went to steering the agent when the goal was precision.